

CS 6263 NLP and Agentic AI: Final Project Rubric

Disciplined Software Development with Agentic LLMs

Version 1.4 | Instructor: Dr. Peyman Najafirad | TA: Mohammad Bahrami Karkevandi | **Submission deadline: May 10, 11:59 PM**

Philosophy: From Vibe Coding to Disciplined Engineering

Agentic LLMs have changed what it means to write software. A capable model can generate working code from a paragraph of intent. This is powerful, and it is also dangerous. The dangerous version is what the community calls vibe coding: a developer types a vague description into an LLM, accepts whatever comes back, runs it once, and moves on. The code may work. The developer may not know why. There is no specification, no tests tied to requirements, no plan that survives the first surprise.

Vibe coding does not scale. It does not survive code review, on-call rotations, security audits, customer escalations, or a teammate trying to extend the codebase six months later. It also does not survive automated regeneration: ask an LLM to rebuild the system from a vague description and you get a different system every time. The research community now treats specification driven development as the discipline that keeps agentic LLMs trustworthy in production.

This project asks you to practice that discipline end to end. The rubric is organized around the six pillars of the software development lifecycle. Each pillar pairs the engineering practice with the artifact you produce, and the test that verifies you produced something real and not theatrical.

The six pillars

Plan. Decide what you are building and for whom, before you write code. In this rubric: a Specification document with public interfaces, and User Stories with acceptance criteria and manual walkthrough steps. The TA verifies your plan is real by feeding it to an LLM and checking that the regenerated code passes your story tests.

Design. Decide how the system will be reproducible by anyone but you. In this rubric: pinned versions, documented data and model provenance, a single command full replay. The TA runs that one command on a clean machine and checks your reported numbers come back.

Implement. Write code that other engineers can read and that does not introduce known vulnerabilities. In this rubric: linting, formatting, type checking, dependency scanning, and a model card that names the system's limits. The TA runs the same checks and reads the model card.

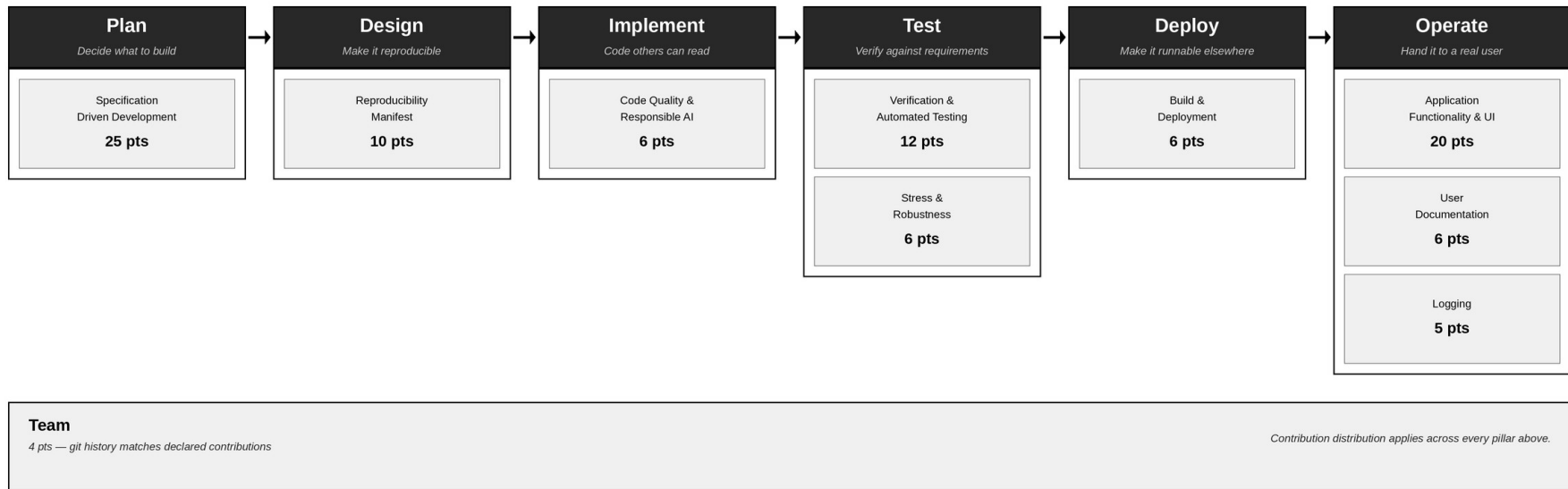
Test. Verify behavior against requirements automatically. In this rubric: unit, integration, user story acceptance, edge, and load tests, all green, with coverage at threshold. Every user story has both an automated test and a manual walkthrough; the two together are how requirements become contracts.

Deploy. Make the system runnable by someone who does not have your laptop. In this rubric: a Dockerfile, a Compose file, a documented environment contract, and a quick start in the README that the TA executes literally. Zero undocumented manual steps.

Operate. Hand a working system to a real user and let them use it. In this rubric: structured logs the TA can trace, a UI a TA can walk through using only your STORIES document, and documentation that a cold reader can follow without asking the team. This is where most points live, because this is where users live.

Six Pillars of the Software Development Lifecycle

Plan → Design → Implement → Test → Deploy → Operate



Total: 100 points

The diagram above shows the same point distribution that the rubric table below uses. The two tallest pillars by point weight are Plan (25 points for Specification Driven Development) and Operate (31 points across Application Functionality, User Documentation, and Logging). Together they account for over half of the grade.

A separate Team category measures how the work was distributed across team members, verified against your git history. The full point breakdown is in the table below.

What this project teaches you

Working with agentic LLMs as a force multiplier requires the same disciplines that make any software system trustworthy: a written contract before code, a reproducible build, a real test suite, a clear deployment story, observable runtime, and accessible documentation. The rubric makes those disciplines countable. Each TEST row is an executable check that distinguishes work from theater.

If you find yourself tempted to write code first and document later, notice what happens at grading time: the spec regeneration test feeds your spec to the same kind of LLM you used while building. If your spec is decoration, regeneration produces a system that fails your own user story tests, and the largest single category in the rubric scores zero. The shortest path to a good grade is also the shortest path to a system you would actually deploy.

How Grading Works

The TA clones your public GitHub repository on a clean machine and works through three phases. About two thirds of the points are computed by automated scripts; the rest comes from a manual walkthrough against your running app. You can run all of these checks yourself with `scripts/preflight.sh` before pushing, and your local result is exactly what the TA will see.

Phase 1. Automated checks (69 pts). make reproduce, make test, make lint, pip-audit, make loadtest, scripts/regenerate.sh. Each writes a JUnit XML or text report under reports/. The grading script reads those reports and scores Plan, Design, Test (both categories), Implement, and Team.

Phase 2. Bring up the system (5 pts for Logging trace). docker compose up reaches healthy under 10 minutes. The TA traces one live request through docker compose logs by request ID; this scores the Logging category.

Phase 3. Manual walkthrough (26 pts). The TA opens docs/STORIES.md and walks every story step by step against the live UI, recording pass or fail per story. The TA also confirms the README cold-read works and that every story has a usage section. This scores App Functionality (20) and Documentation (6).

Key Terms

Spec regeneration. Feeding docs/SPEC.md to an LLM with the course-issued prompt, then running your user story tests against the generated code. If the spec is the real contract, the generated code passes; if the spec is decoration, it does not.

User story. A short feature description from the user's perspective with a stable ID (US-NN), Given/When/Then acceptance criteria, and numbered manual steps a TA can follow without reading source. Stored in docs/STORIES.md.

Manual walkthrough. The TA opens STORIES.md and follows each story step by step against the running app, recording pass or fail in reports/walkthrough.md. This is how App Functionality is graded.

Traceability matrix. grading/traceability.yaml. Links each user story to its spec section, source modules, and tests, so the grader can detect orphans.

Preflight. scripts/preflight.sh. Runs every automated check the TA will run. If preflight passes locally, your automated grade passes. The manual walkthrough still requires a human.

Pillar	Category	TEST + Required Artifacts	Points
Plan	Specification Driven Development	TEST: Spec regeneration The TA runs scripts/regenerate.sh. The script feeds docs/SPEC.md to an LLM (model claude-opus-4-5-20251101, temperature 0, course-issued prompt at scripts/regenerate_prompt.md) and asks it to generate the source code. The TA then runs the user story acceptance tests against the generated code. Score is proportional: ≥90 percent passing earns full credit, <50 percent scores zero, between is proportional. This is the largest single category in the rubric and the no-vibe-coding test: if the spec is the real source of truth, generation produces a	25

Pillar	Category	TEST + Required Artifacts	Points
		working system; if the spec is decoration, it does not. <i>Required artifacts:</i> - docs/SPEC.md with sections: Purpose, Component Inventory (name to module path), Data Flow, Public Interfaces, Model and Prompt Selection - docs/STORIES.md with stable IDs (US-NN), Given/When/Then, numbered manual steps, and reference screenshots - docs/diagrams/architecture.png or .svg plus its source file - grading/traceability.yaml mapping every story to spec section, source modules, and tests - scripts/regenerate.sh runnable by TA on a clean machine	
Design	Reproducibility Manifest	TEST: One command full replay The TA runs make reproduce on a clean machine. The target rebuilds the Docker environment, downloads data and models, runs the pipeline, runs all tests, and regenerates the headline numbers reported in the README. Full credit if every step succeeds and metrics are within the documented tolerance; half credit if pipeline completes but metrics drift; zero if make reproduce fails to complete. <i>Required artifacts:</i> - grading/manifest.yaml pinning Python version, seed, model IDs, dataset versions, commit_sha - docs/DATA.md and docs/MODELS.md documenting source, version or hash, license per item - docs/REPRODUCE.md with hardware profile, expected runtime, expected metric values plus tolerance - requirements.txt or pyproject.toml with pinned versions (no floating tags) - make reproduce, make download-data, make download-models targets	10
Deploy	Build and Deployment	TEST: Zero touch fresh clone build The TA does a fresh git clone, copies .env.example to .env and fills only the placeholders flagged required, then runs docker compose up. Wall clock to all services healthy must be under 10 minutes. Each undocumented manual step (extra pip install, manual model download, missing migration command) loses 1 point. Full credit if zero manual steps needed. <i>Required artifacts:</i> - Dockerfile (multi stage preferred, runs as non root user) - docker-compose.yml with health checks on every long running service .env.example with placeholders and comments; .env in .gitignore; no real secrets in repo history	6

Pillar	Category	TEST + Required Artifacts	Points
		Quick start in README.md is the literal command sequence the TA will run	
Test	Verification and Automated Testing	TEST: make test green with coverage and reports The TA runs make test inside the Docker container. Score: 3 pts for make test exit 0; 2 pts for all three JUnit XML reports present and parseable (unit, integration, user_stories); 2 pts for coverage at or above 70 percent on business logic; 5 pts for user story test pass rate at or above 90 percent (proportional below). <i>Required artifacts:</i> - tests/unit/, tests/integration/, tests/user_stories/ with one test per source module and per story - Pytest marker @pytest.mark.user_story("US-NN") on each story test, with Given/When/Then in the docstring - reports/unit.xml, reports/integration.xml, reports/user_stories.xml (JUnit XML) - reports/coverage.xml and reports/coverage_html/ regenerated by the test run - scripts/demo.sh or make demo exercising every user story end-to-end	12
Test	Stress and Robustness	TEST: Live load test on TA machine The TA runs the team's load test against the live docker compose up system on the TA's hardware. System must sustain at least 10 requests per second with under 5 percent error rate over a 60 second window on the headline endpoint (4 pts). The edge case suite must complete with zero crashes (2 pts). <i>Required artifacts:</i> - tests/load/locustfile.py (or k6 or wrk equivalent) runnable as make loadtest - tests/edge/ covering empty, very long, non-ASCII, multilingual, code-mixed, and adversarial inputs - docs/benchmarks.md with hardware section, methodology, and headline numbers - reports/benchmarks.json with raw timings	6
Implement	Code Quality and Responsible AI	TEST: make lint clean, pip-audit clean, model card present The TA runs make lint and pip-audit on a clean checkout, and inspects docs/MODEL_CARD.md. Score: 3 pts for make lint exit 0; 1 pt for pip-audit showing no Critical or High severity vulnerabilities unaddressed (Medium and below do not block); 2 pts for MODEL_CARD.md containing all four required sections with substantive content. <i>Required artifacts:</i> ruff plus black plus mypy configured in pyproject.toml; make lint target	6

Pillar	Category	TEST + Required Artifacts	Points
		- reports/security.txt from pip-audit; any flagged vulnerability has mitigation rationale - docs/MODEL_CARD.md sections: Intended Use, Limitations, Risks, Out of Scope	
Operate	Logging	TEST: Trace one request end to end The TA picks any UI action (or API request) on the running system and traces it end to end through docker compose logs -f app using its request ID. Full credit (5 pts) if every component the request touches produces a log line tagged with the same request ID and the TA can follow input arrival, model call, response generation, and output delivery from the logs alone. Half credit (2.5 pts) for partial traces. Zero if request ID is missing or logs are unstructured. <i>Required artifacts:</i> - Structured JSON logging; every entry has timestamp, level, module, request_id; no bare print() in production paths - Request ID propagated through every component - docs/LOGGING.md with a worked example showing a captured request ID and the log lines that follow it	5
Operate	Application Functionality and UI	TEST: Per story manual walkthrough After the TA brings up the system with docker compose up, they open docs/STORIES.md and walk every story step by step against the live UI. Each story passes only when every numbered step works as written and the end state matches the reference screenshot. Score is proportional: (stories passed / total stories) times 20, rounded to nearest 0.5. The TA records each story result in reports/walkthrough.md with a one line observation. <i>Required artifacts:</i> - docs/STORIES.md is followable by a human without reading source code (numbered steps, exact inputs, exact observable outcomes) - Every story reachable from the running docker compose up system - At least 2 stories exercise error paths (clearly marked) with documented expected error messages - docs/assets/stories/us_NN_expected.png per story - reports/walkthrough.md (TA fills in during grading)	20
Operate	User Documentation	TEST: Cold read documentation walkthrough The TA performs three checks: (1) reads only README.md and follows quick start cold from a fresh clone (3 pts; -1 per undocumented manual fix the TA discovers); (2) verifies every story in STORIES.md has a corresponding section in docs/usage.md (2 pts); (3) compares each us_NN_expected.png to the live UI during the App Functionality walkthrough (1 pt).	6

Pillar	Category	TEST + Required Artifacts	Points
		<i>Required artifacts:</i> • README.md with title, description, tech stack, quick start, and Results section declaring headline metrics with tolerance (e.g. accuracy 0.85 ± 0.02); source layout <code>src/<package_name>/</code>, tests under <code>tests/</code> • docs/usage.md with one section per story • docs/assets/stories/us_NN_expected.png per story, plus docs/assets/demo.gif or linked video	
Team	Team Contributions	TEST: git shortlog matches declared percentages The TA runs <code>git shortlog -sne --all --no-merges</code> and compares the actual commit distribution to CONTRIBUTIONS.md. Score: 2 pts if each member's commit share is within ± 15 percentage points of declared share; 2 pts if each member has commits across at least two of: source, tests, docs. <i>Required artifacts:</i> • CONTRIBUTIONS.md listing each member with role, modules owned, and percent contribution summing to 100 • reports/git_contributions.txt from <code>git shortlog -sne --all --no-merges</code>	4
TOTAL			100

How to Submit

- Push the entire project to a public GitHub repository owned by your team. Submit the repository URL through the course submission system on or before **May 10, 11:59 PM**. The link must point to a public repo so the TA can clone without credentials. Repos that are private, deleted, or made private after submission score zero.
- The grading script reads `commit_sha` from `grading/manifest.yaml`. If missing or stale, the TA grades the latest commit on main as of the deadline.
- Run `scripts/preflight.sh` locally before pushing. If preflight fails, your grade fails the same way.
- Late submissions follow the course-wide late policy. Contact the instructor before the deadline if you need an extension; post-hoc explanations are not accepted.